

UART Receiver

Design Implementation and Verification Report

Contents

1	Introduction	2
2	Block Interface	2
2.1	Top-Level I/O Diagram	2
3	RTL Architecture	3
3.1	Top-Level UART_RX Block Diagram	3
3.2	Design Notes – Output Timing Behavior	3
4	Sub-Module Descriptions	4
4.1	Control Unit – Rx_FSM	4
4.2	Input Sampling – sampler	4
4.3	Timing Reference – edge_bit_counter	4
4.4	Parity Computation – parity_checker	4
4.5	Shift Register – deserializer	4
5	Verification Methodology	5
5.1	Testbench	5
5.2	Waveform Verification	6
5.3	Simulation Log Results	14

1 Introduction

A Universal Asynchronous Receiver/Transmitter (UART) is a block of circuitry responsible for implementing full-duplex serial communication.

Receiving UART (UART_RX) recovers a serial bitstream sampled on a single input line and reconstructs it into a parallel data byte, following the standard UART framing convention:

START -- DATA[7:0] -- (optional PARITY) -- STOP

This report documents the RTL implementation of the UART_RX block, and the self-checking testbench used to validate the design against the specification.

2 Block Interface

The top-level UART_RX module exposes the following ports, as defined by the specification:

Port	Width	Description
CLK	1	UART RX clock signal
RST	1	Asynchronous active-low reset
RX_IN	1	Serial data input line
Prescale	6	Oversampling ratio (4 / 8 / 16 / 32)
PAR_EN	1	Parity enable
PAR_TYP	1	Parity type (0 = even, 1 = odd)
P_DATA	8	Output deserialized data byte
data_valid	1	Output valid strobe (high for 1 cycle)
parity_error	1	Output, asserted on parity mismatch
stop_error	1	Output, asserted on invalid stop bit

Table 1: UART_RX port list

2.1 Top-Level I/O Diagram

Figure 1 shows the black-box view of the UART_RX module, grouping the configuration inputs (Prescale, PAR_EN, PAR_TYP) separately from the data and control signals.

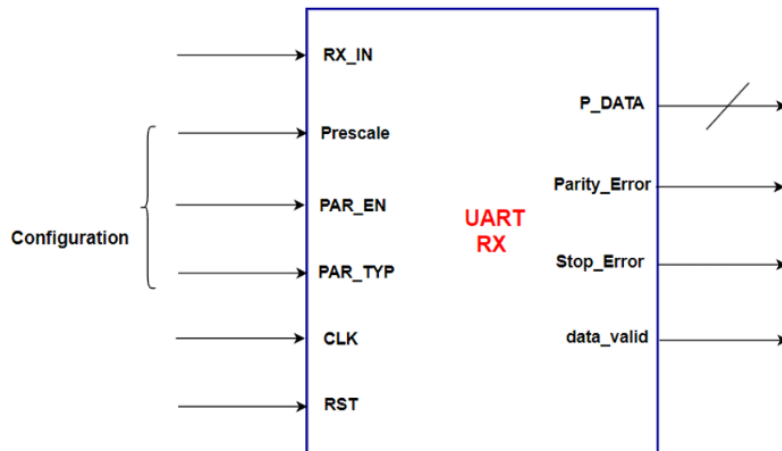


Figure 1: UART_RX top-level I/O diagram

3 RTL Architecture

This section presents the structural partitioning of the design and confirms that each sub-block performs its intended function.

3.1 Top-Level UART_RX Block Diagram

Figure 2 shows the top-level interconnection. `samp_inst` and `cnt_inst` provide sampled and timing signals to `fsm_inst`, which in turn drives the enable signals for `par_inst` and `deser_inst`. This confirms the structural partitioning of the datapath around the central control FSM.

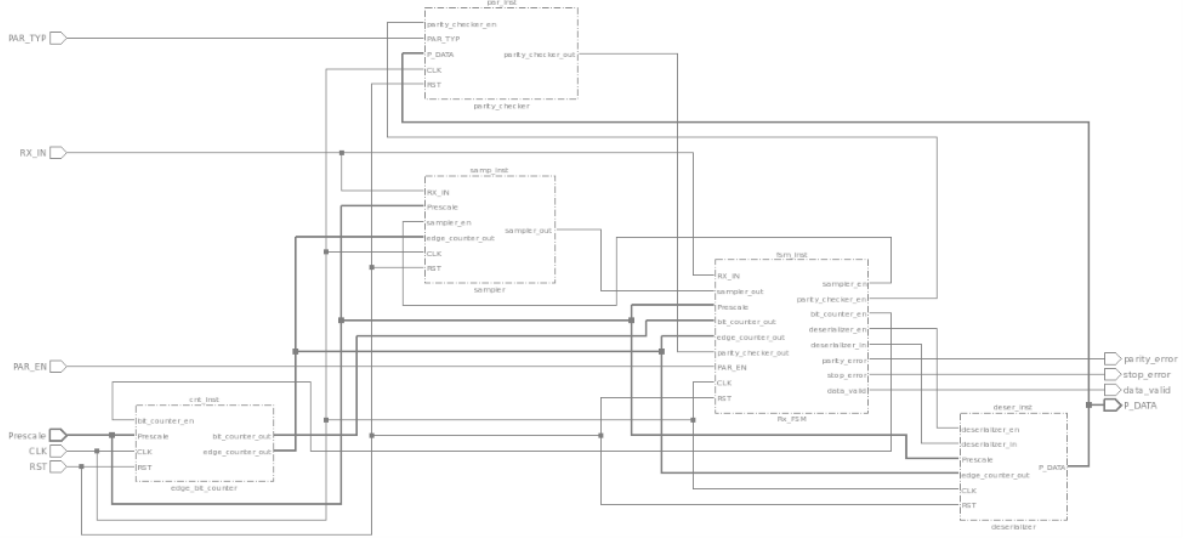


Figure 2: Top-level block diagram of the UART_RX module

3.2 Design Notes – Output Timing Behavior

The following timing behavior is intentional and is visible in every waveform capture in Section 5.2:

- `data_valid` is combinationaly driven high for exactly one clock cycle while the FSM is in the `FRAME_END` state, immediately after the stop bit has been validated. It is not registered beyond that state, so it naturally deasserts as soon as the FSM transitions out of `FRAME_END`.
- `parity_error` is asserted only during the last clock cycle of the `CHECK_PARITY_BIT` state – i.e. the cycle where `edge_counter_out == Prescale-1`, which is when the parity comparison against `sampler_out` is actually evaluated. on that final qualifying cycle.
- `stop_error` follows the same pattern: it is asserted only during the last clock cycle of the `CHECK_STOP_BIT` state (`edge_counter_out == Prescale-1`), the cycle at which the stop bit is sampled and checked.

4 Sub-Module Descriptions

4.1 Control Unit – Rx_FSM

A FSM with six states governs the reception sequence: `IDLE` → `CHECK_START_BIT` → `DATA_BITS` → `CHECK_PARITY_BIT` (optional) → `CHECK_STOP_BIT` → `FRAME_END`.

State	Function
<code>IDLE</code>	Waits for <code>RX_IN</code> to fall (potential start bit)
<code>CHECK_START_BIT</code>	Confirms start bit is still low at mid-bit sample point
<code>DATA_BITS</code>	Shifts in <code>DATA_WIDTH</code> data bits via the deserializer
<code>CHECK_PARITY_BIT</code>	Samples and compares parity bit against computed parity
<code>CHECK_STOP_BIT</code>	Confirms stop bit is high; flags <code>stop_error</code> otherwise
<code>FRAME_END</code>	Pulses <code>data_valid</code> ; checks for back-to-back frame

Table 2: Rx_FSM state description

4.2 Input Sampling – sampler

Implements two selectable sampling strategies (via the `sampling_methodology` parameter):

- **Mid-bit single-point sampling:** captures `RX_IN` once at the bit-period midpoint.
- **Majority-vote sampling** (default): takes three samples straddling the midpoint and outputs the majority value, rejecting single-sample glitches.

4.3 Timing Reference – edge_bit_counter

Generates two counters from `CLK`:

- `edge_counter_out`: counts clock ticks within the current bit period (rolls over at `Prescale`).
- `bit_counter_out`: counts completed data bits within the current frame, enabled only during `DATA_BITS`.

4.4 Parity Computation – parity_checker

Computes parity over `P_DATA` using XOR-reduction, selecting even (`^P_DATA`) or odd (`~(^P_DATA)`) based on `PAR_TYP`.

4.5 Shift Register – deserializer

Right-shifts `deserializer_in` into `P_DATA[DATA_WIDTH-1]` once per bit period (when `edge_counter_out == Prescale-1`), so the first received bit ends up at `P_DATA[0]`.

5 Verification Methodology

5.1 Testbench

A self-checking testbench (`uart_rx_tb`) drives the `UART_RX` DUT with a clock derived from a frequency equals to 115200 Hz equivalent bit period of 8680.56, scaled by the configured `Prescale` value to generate the correct oversampling clock.

The testbench is built around three reusable tasks:

- **Reset_UART** – drives `RX_IN_tb` idle high, clears `PAR_EN_tb`/`PAR_TYP_tb`, sets the default `Prescale_tb = 8`, asserts for one bit period, then releases it.
- **set_prescale** – updates `Prescale_tb` and recomputes the RX clock half-period accordingly, allowing the same stimulus tasks to exercise multiple oversampling ratios.
- **receive_frame** – parameterized by the start bit, 8-bit data byte, `PAR_EN`, `PAR_TYP`, parity bit, stop bit, and `Prescale`, plus the expected `P_DATA`, `data_valid`, `parity_error`, and `stop_error`.

It serially drives the complete frame onto `RX_IN_tb` bit-by-bit, then self-checks the DUT outputs against the expected values and prints a `PASSED/FAILED` verdict via `$display`.

A combinational flag-latching block continuously samples `data_valid_tb`, `parity_error_tb`, and `stop_error_tb` into “seen” flags so that single-cycle pulses are not missed regardless of when the checker samples them.

The main stimulus block resets the DUT and then exercises 22 scenarios, covering:

1. Basic no-parity reception (Cases 1–2, 5–6): normal, all-zero, and all-one data patterns.
2. Even- and odd-parity reception with correct parity (Cases 3–4, 7–8).
3. Framing and parity error detection (Cases 9–11): stop bit held low, and both even/odd parity mismatches.
4. Alternate oversampling ratios (Cases 12–14, 19–20): `Prescale = 4, 16, and 32`.
5. Back-to-back frame reception with no idle gap (Cases 15–16).
6. Error recovery (Cases 17–18): a framing error immediately followed by a valid frame, confirming the FSM returns cleanly to `IDLE`.
7. A deep-idle test (Case 21) inserting three full bit periods of idle line before a normal frame.
8. A final scenario coded directly in the stimulus block (not via the task) to test start-bit glitch rejection (Case 22): `RX_IN` is pulled low for only two RX clock ticks (less than half a bit period) and released back high before the mid-bit validation point, verifying the FSM remains in `IDLE` and asserts none of `data_valid`, `parity_error`, or `stop_error`.

5.2 Waveform Verification

Figures 3–24 show ModelSim waveform captures of RX_IN.tb, P_DATA.tb, data_valid.tb, parity_error.tb, stop_error.tb, and the FSM current_state/next_state signals for each of the 22 test cases.

Note: the waveform grids below have been edited so that a visible gap equal to one TX bit period is shown between any two consecutive lines/frames in the capture, making the frame boundaries and inter-frame idle time easier to identify at a glance.

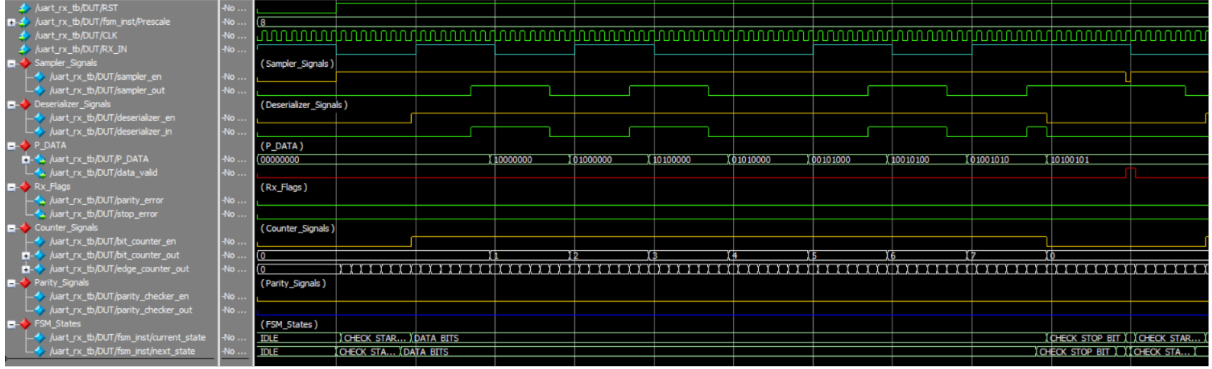


Figure 3: Test Case 1: no parity, data = 10100101

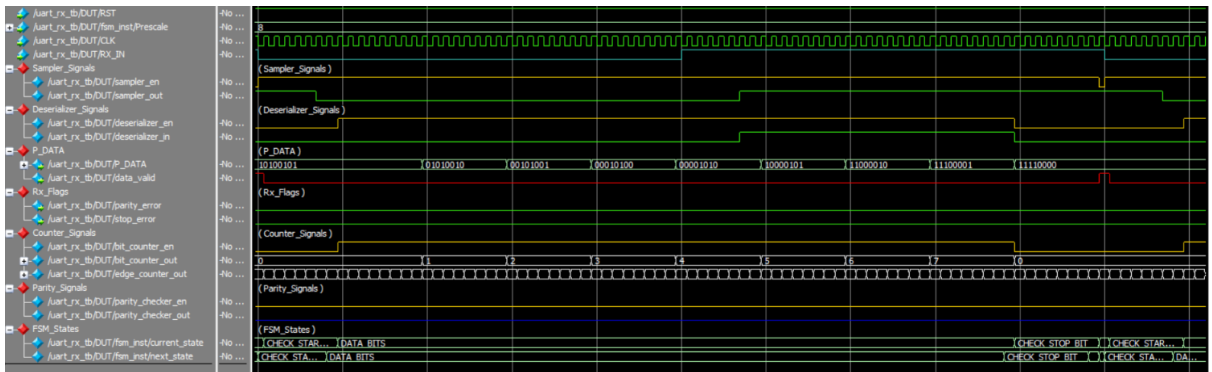


Figure 4: Test Case 2: no parity, data = 11110000

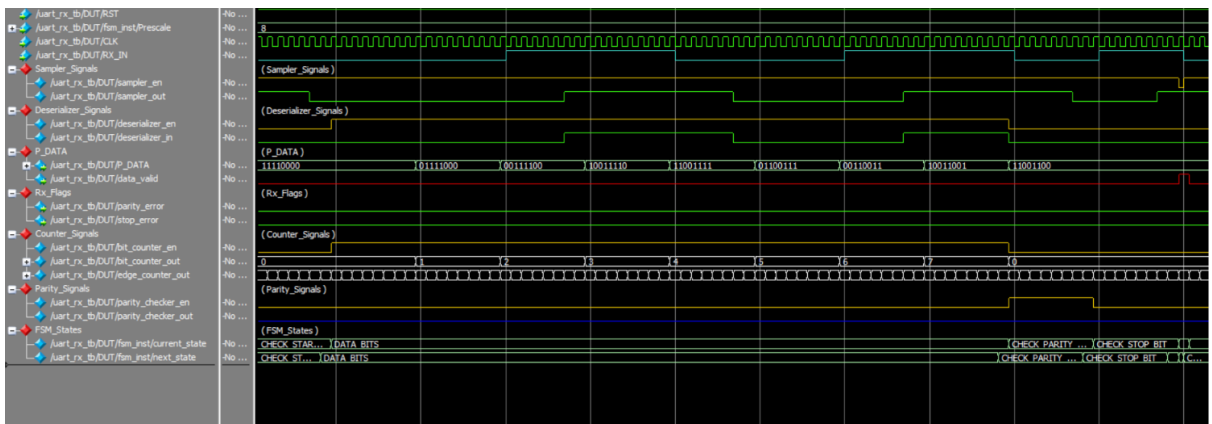


Figure 5: Test Case 3: even parity, correct

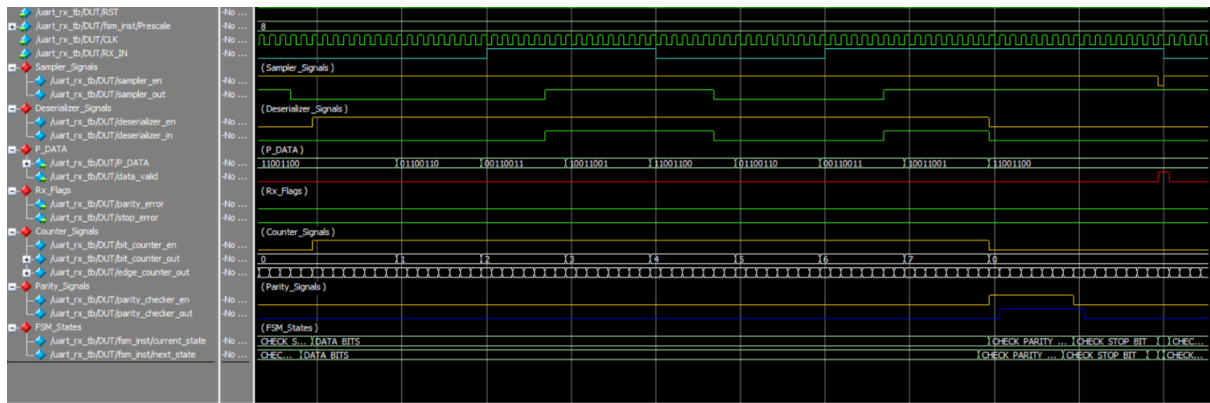


Figure 6: Test Case 4: odd parity, correct

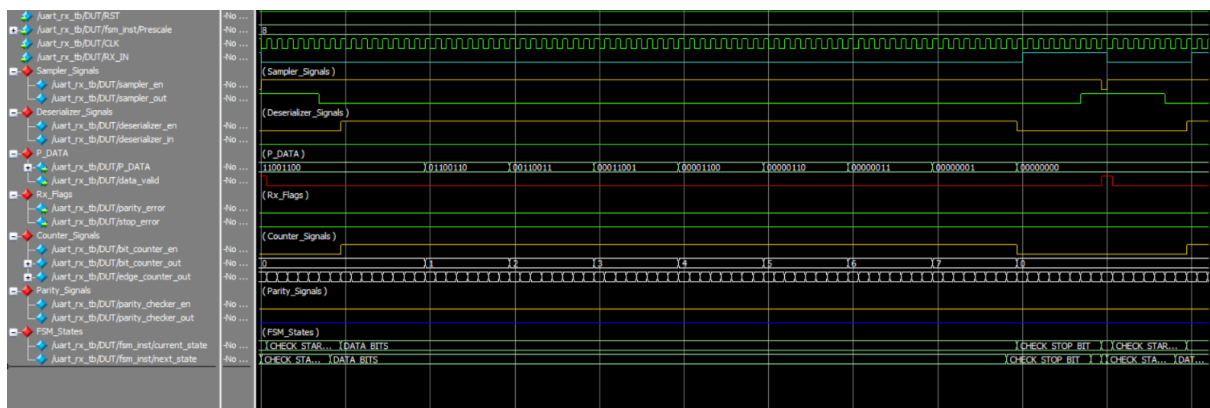


Figure 7: Test Case 5: all zeros

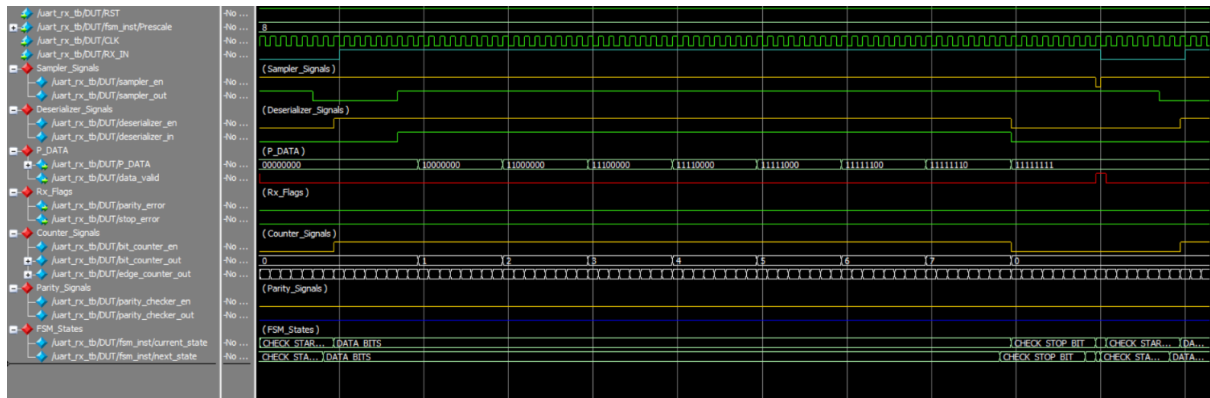


Figure 8: Test Case 6: all ones

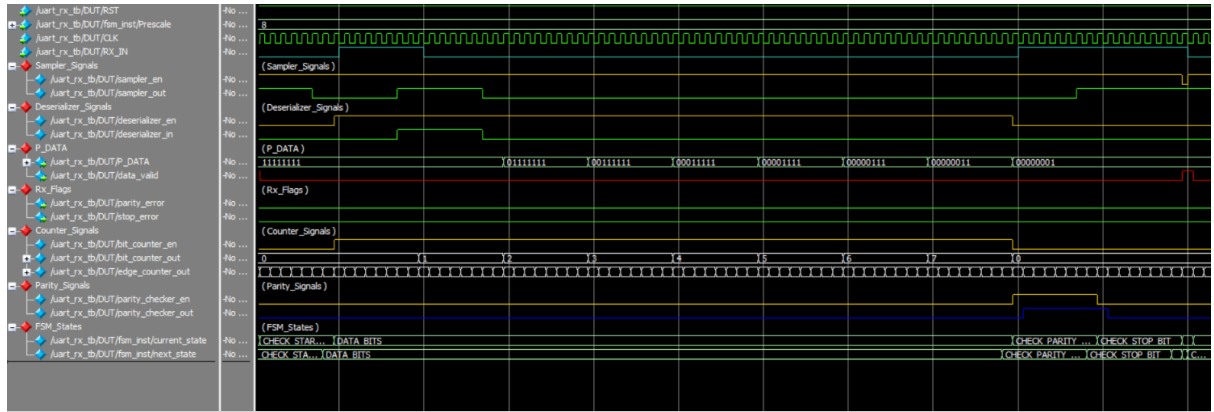


Figure 9: Test Case 7: single LSB, even parity

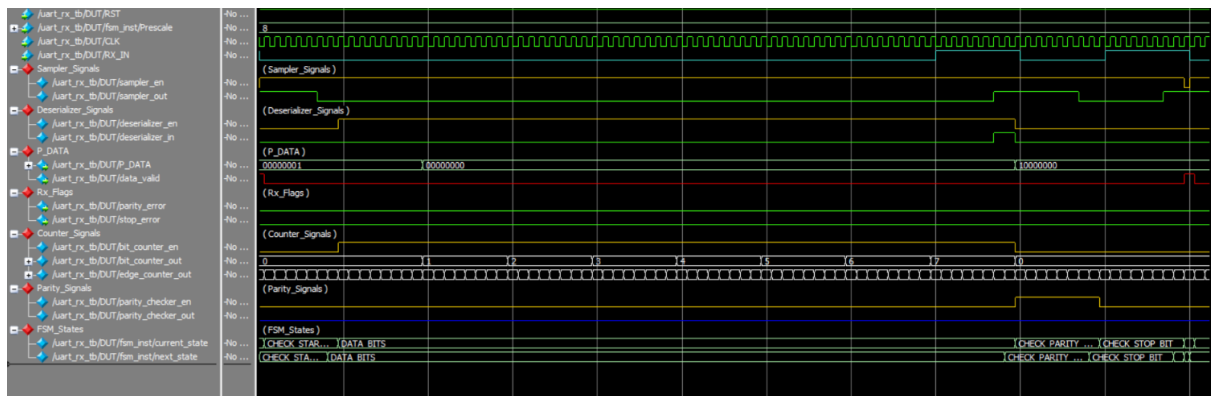


Figure 10: Test Case 8: single MSB, odd parity

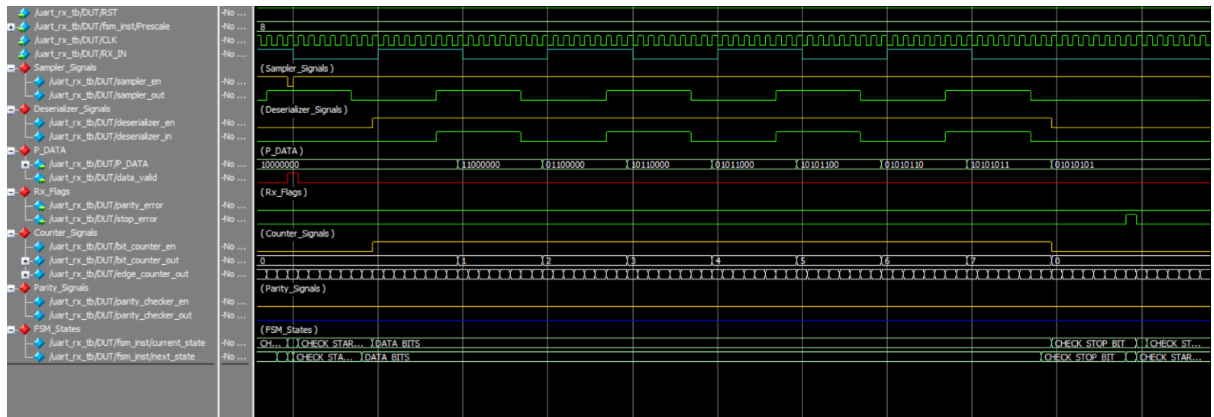


Figure 11: Test Case 9: stop error

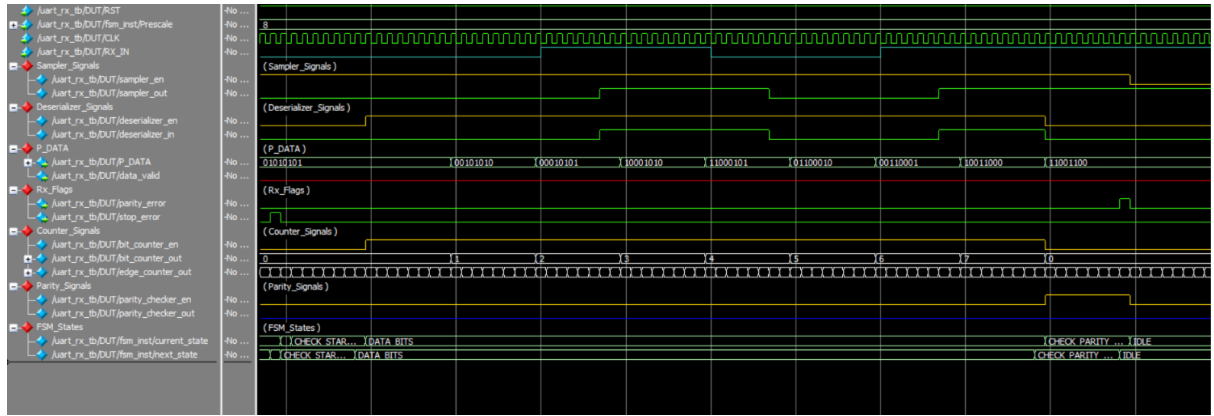


Figure 12: Test Case 10: even parity error

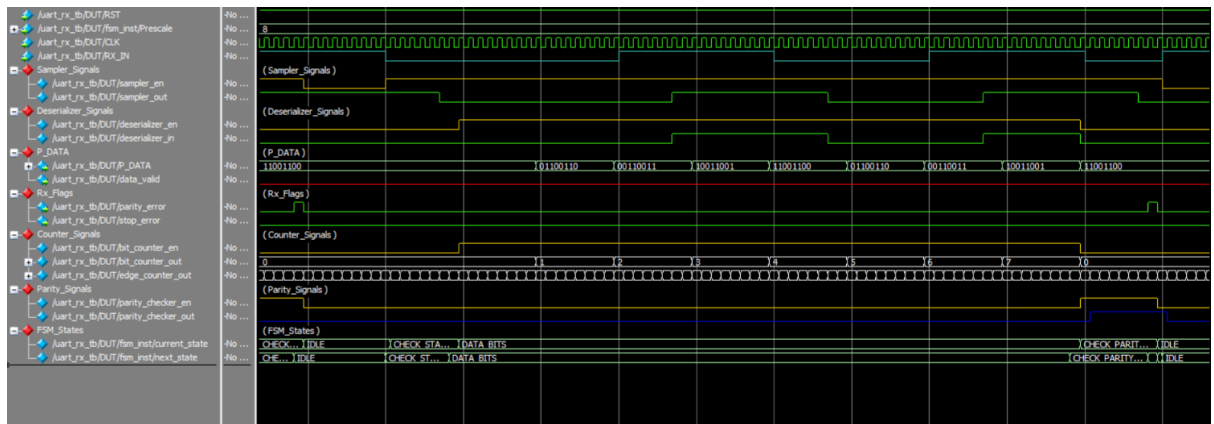


Figure 13: Test Case 11: odd parity error

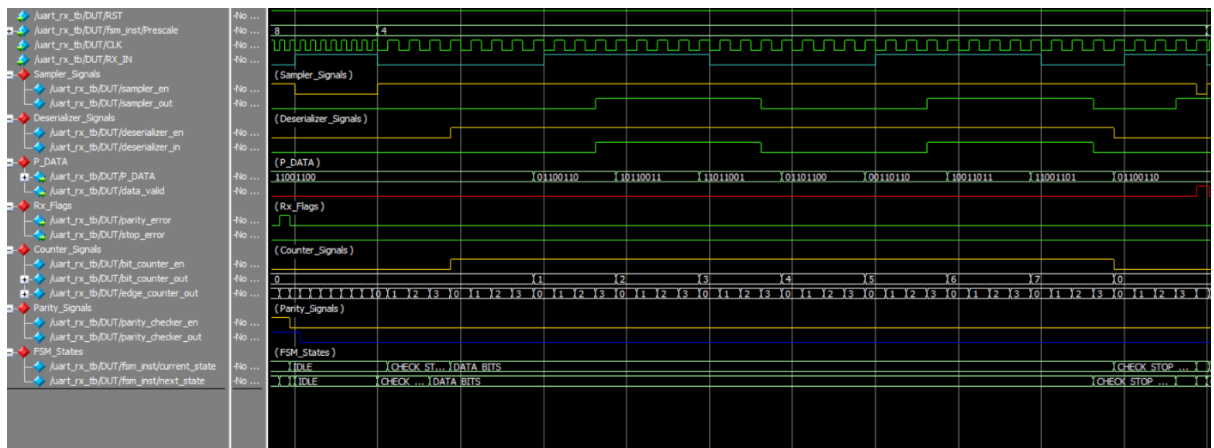


Figure 14: Test Case 12: Prescale = 4

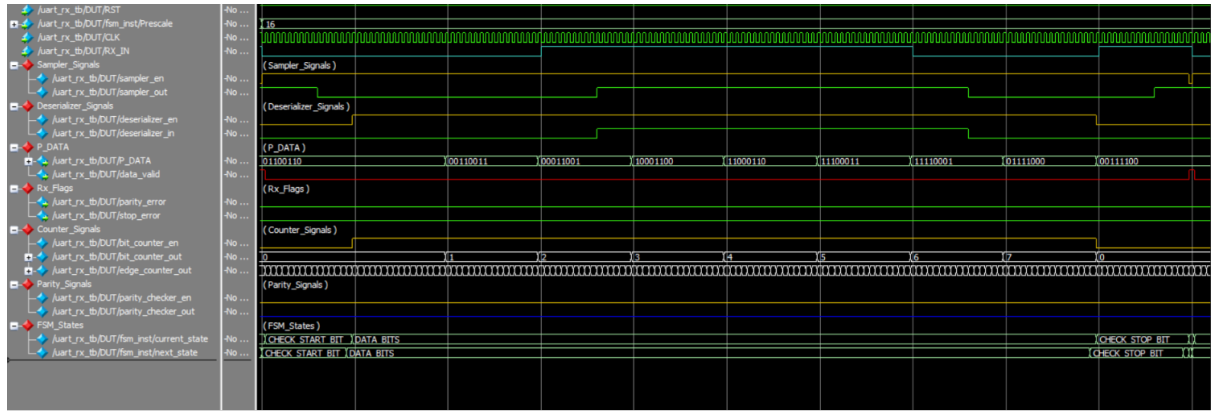


Figure 15: Test Case 13: Prescale = 16

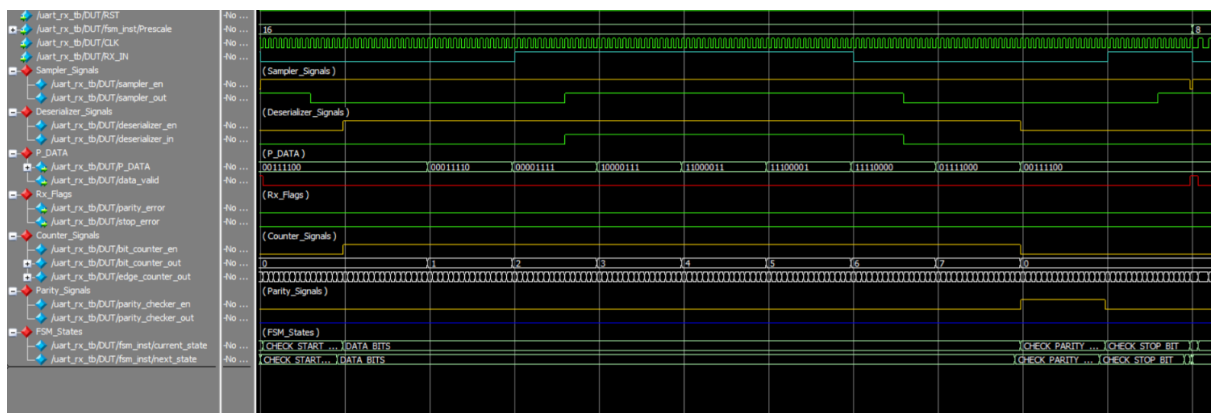


Figure 16: Test Case 14: Prescale = 16, even parity

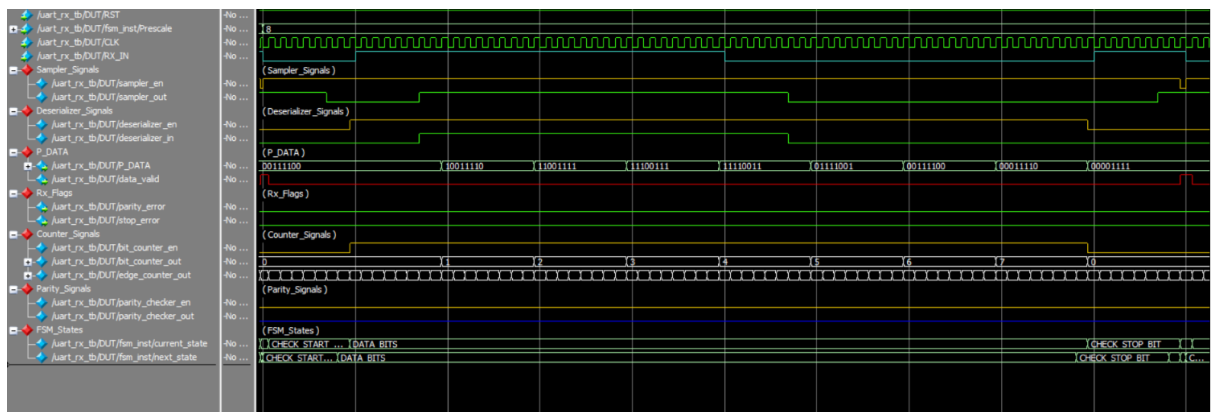


Figure 17: Test Case 15: back-to-back frame 1

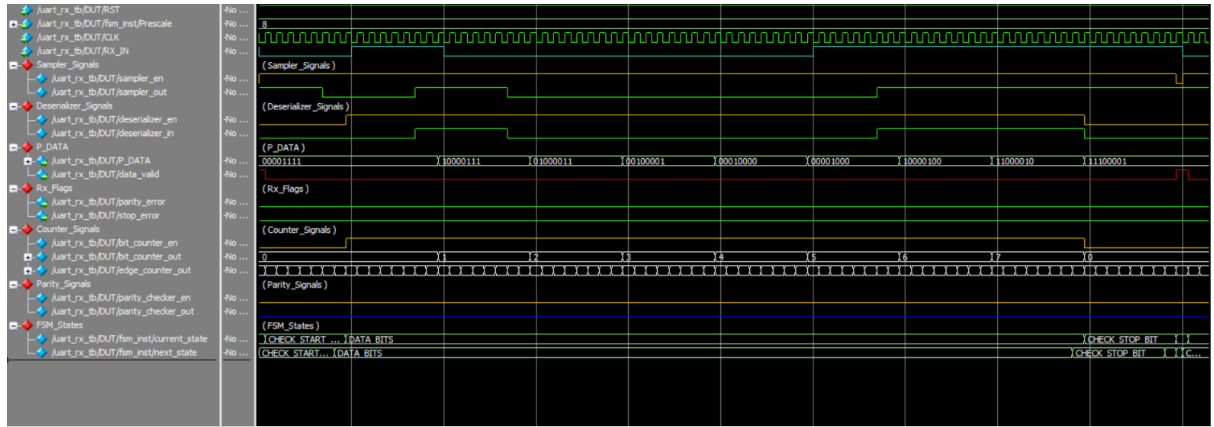


Figure 18: Test Case 16: back-to-back frame 2

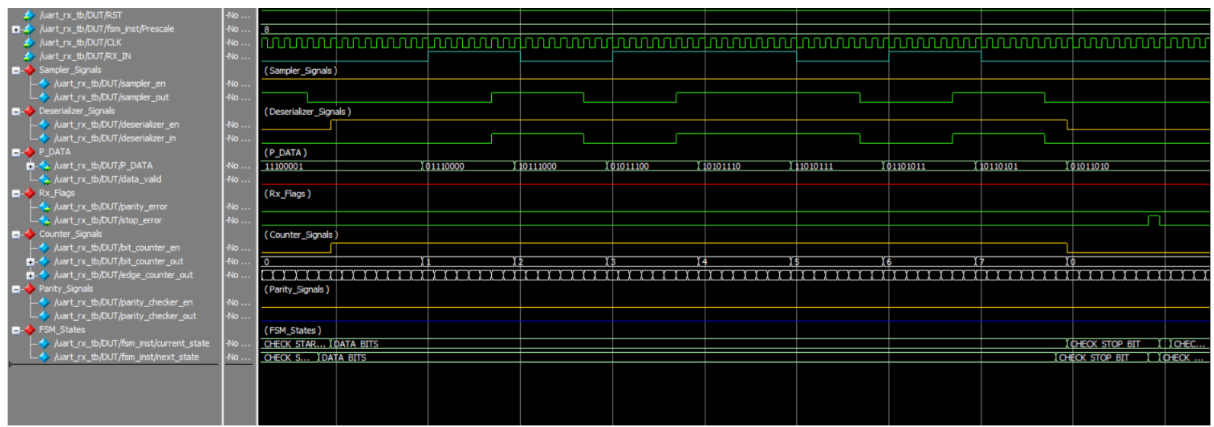


Figure 19: Test Case 17: stop error recovery test

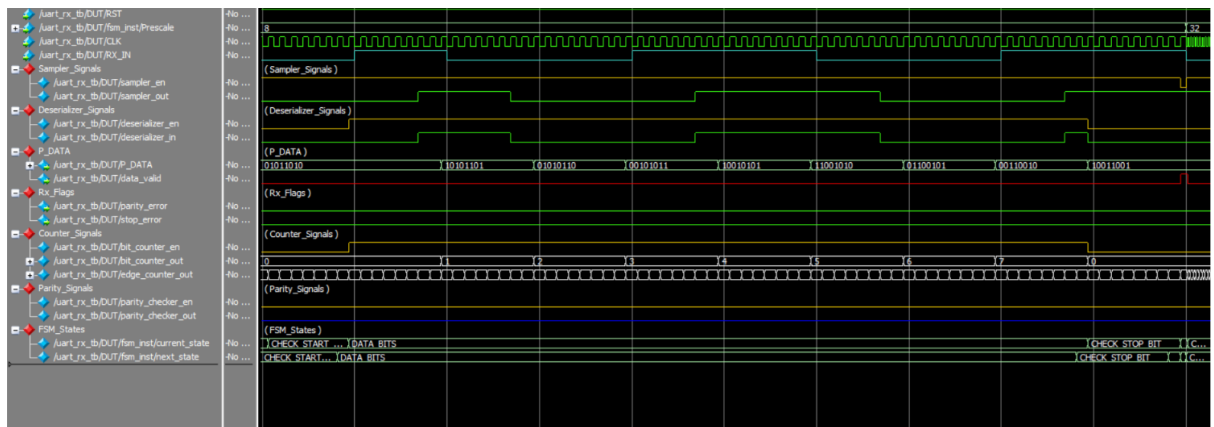


Figure 20: Test Case 18: valid frame after error

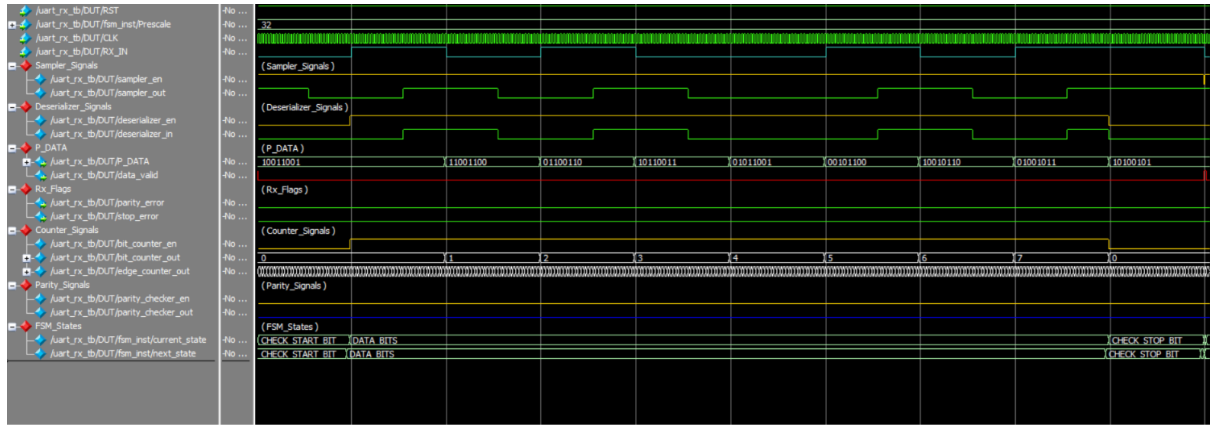


Figure 21: Test Case 19: Prescale = 32

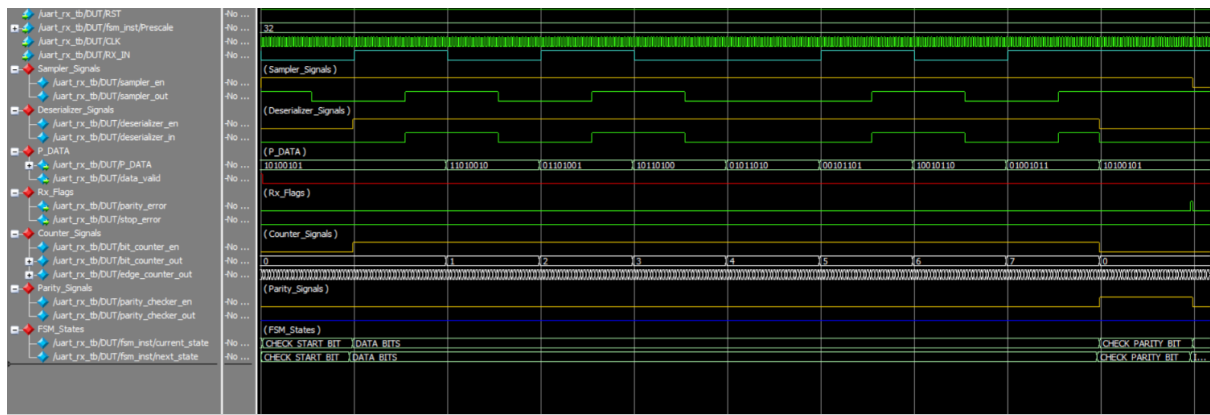


Figure 22: Test Case 20: Prescale = 32, parity error

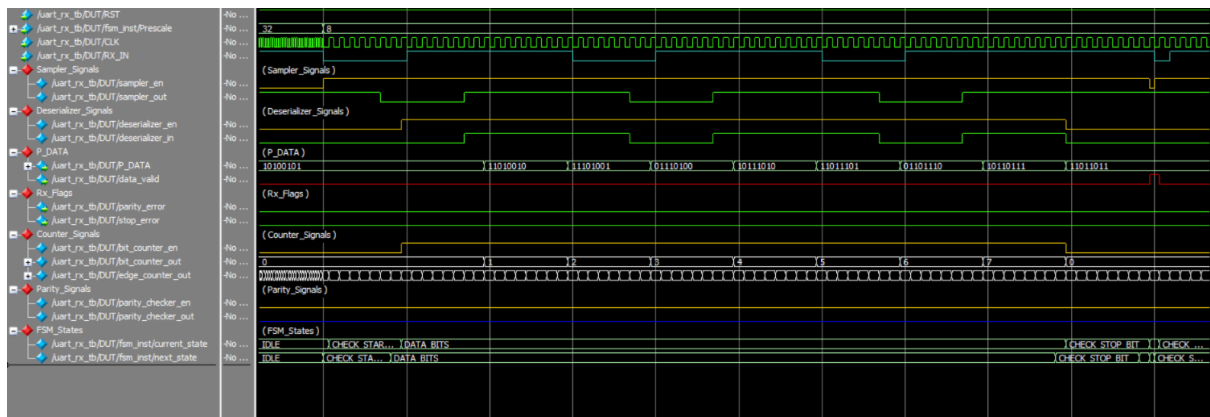


Figure 23: Test Case 21: deep idle test

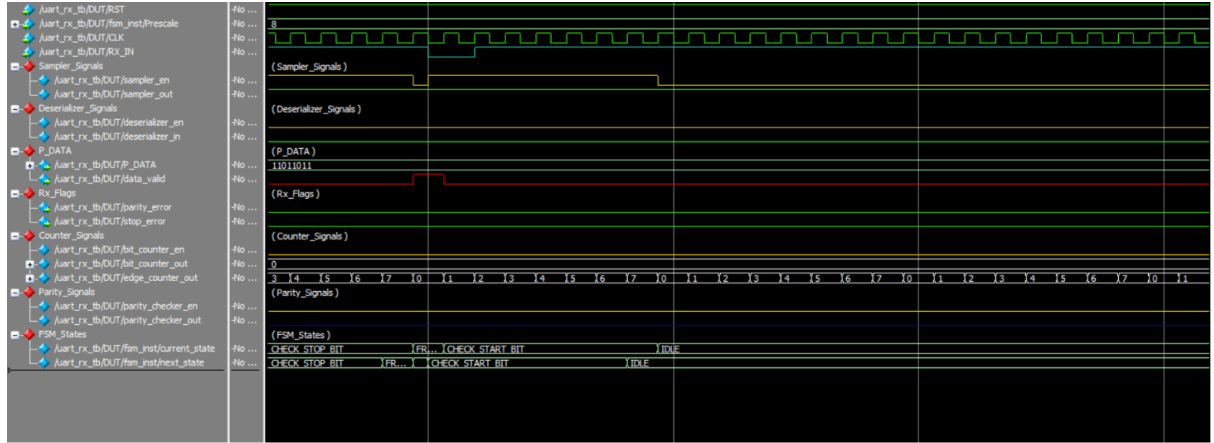


Figure 24: Test Case 22: false start-bit glitch rejection

In every capture, the FSM correctly sequences through IDLE → CHECK_START_BIT → DATA_BITS → CHECK_PARITY_BIT (when enabled) → CHECK_STOP_BIT → FRAME_END, with data_valid_tb pulsing for one cycle during FRAME_END on success, and parity_error_tb/stop_error_tb pulsing for one cycle during the last cycle of CHECK_PARITY_BIT/CHECK_STOP_BIT on the corresponding fault-injection cases, consistent with the timing behavior described in Section 3.2.

5.3 Simulation Log Results

The console transcript below captures the self-checking testbench output for all 22 test cases described in Section 5.1. Each line reports the transmitted bit pattern, the recovered P_DATA, the latched data_valid/parity_error/stop_error flags, and a PASS/FAIL verdict.

```
=====
                        RUNNING UART RX TESTS
=====
In: 10100101 | Out: 10100101 | Data_Valid: 1 Parity_Error: 0 Stop_Error: 0 -> PASSED
In: 11110000 | Out: 11110000 | Data_Valid: 1 Parity_Error: 0 Stop_Error: 0 -> PASSED
In: 11001100 | Out: 11001100 | Data_Valid: 1 Parity_Error: 0 Stop_Error: 0 -> PASSED
In: 11001100 | Out: 11001100 | Data_Valid: 1 Parity_Error: 0 Stop_Error: 0 -> PASSED
In: 00000000 | Out: 00000000 | Data_Valid: 1 Parity_Error: 0 Stop_Error: 0 -> PASSED
In: 11111111 | Out: 11111111 | Data_Valid: 1 Parity_Error: 0 Stop_Error: 0 -> PASSED
In: 00000001 | Out: 00000001 | Data_Valid: 1 Parity_Error: 0 Stop_Error: 0 -> PASSED
In: 10000000 | Out: 10000000 | Data_Valid: 1 Parity_Error: 0 Stop_Error: 0 -> PASSED
In: 01010101 | Out: 01010101 | Data_Valid: 0 Parity_Error: 0 Stop_Error: 1 -> PASSED
In: 11001100 | Out: 11001100 | Data_Valid: 0 Parity_Error: 1 Stop_Error: 0 -> PASSED
In: 11001100 | Out: 11001100 | Data_Valid: 0 Parity_Error: 1 Stop_Error: 0 -> PASSED
In: 01100110 | Out: 01100110 | Data_Valid: 1 Parity_Error: 0 Stop_Error: 0 -> PASSED
In: 00111100 | Out: 00111100 | Data_Valid: 1 Parity_Error: 0 Stop_Error: 0 -> PASSED
In: 00111100 | Out: 00111100 | Data_Valid: 1 Parity_Error: 0 Stop_Error: 0 -> PASSED
In: 00001111 | Out: 00001111 | Data_Valid: 1 Parity_Error: 0 Stop_Error: 0 -> PASSED
In: 11100001 | Out: 11100001 | Data_Valid: 1 Parity_Error: 0 Stop_Error: 0 -> PASSED
In: 01011010 | Out: 01011010 | Data_Valid: 0 Parity_Error: 0 Stop_Error: 1 -> PASSED
In: 10011001 | Out: 10011001 | Data_Valid: 1 Parity_Error: 0 Stop_Error: 0 -> PASSED
In: 10100101 | Out: 10100101 | Data_Valid: 1 Parity_Error: 0 Stop_Error: 0 -> PASSED
In: 10100101 | Out: 10100101 | Data_Valid: 0 Parity_Error: 1 Stop_Error: 0 -> PASSED
In: 11011011 | Out: 11011011 | Data_Valid: 1 Parity_Error: 0 Stop_Error: 0 -> PASSED
Glitch ignored successfully. -> PASSED
```